

DeepCheck — Tam Texniki İzah

Real vaxtlı davranış analizi ilə bot aşkarlama sistemi

Teknofest Finansal Teknologiyalar Yarışması · Komanda daxili sənəd

Versiya: cef6dbf · 9 Sentyabr 2026

Bu sənəd layihənin **hər texniki detalları** əhatə edir: nəyi, niyə istifadə etdiyimiz, nəyin işlədiyi və **nəyin işləmədiyi**.

Bu sənədi oxuyan komanda üzvü jürinin verə biləcəyi istənilən texniki suala cavab verə bilməlidir. Ona görə burada təkcə «necə işləyir» yox, «niyə belə qurulub», «hansı alternativini rədd etdik» və «harada uğursuz olur» da var. Sonuncu ən vacibidir: ölçdüyümüz zəiflikləri gizlətmirik, çünki jüri qarşısında ən güclü mövqe ölçülmüş dürüstlükdür.

Mündəricat

- | | |
|---|--------------------------------------|
| 1. Problem və həll ideyası | 10. Təhlükəsizlik qatları |
| 2. Ümumi arxitektura | 11. Verilənlər bazası sxemi |
| 3. Brauzer SDK-sı | 12. Model təlimi |
| 4. API endpointləri | 13. Ölçmələr — real nəticələr |
| 5. 12 öznitelik — hər biri ayrıca | 14. Nə işləmir və niyə |
| 6. Normalizasiya: niyə log-persentil | 15. Testlər, CI, işə salma |
| 7. Üç model və ansambl | 16. Jüri sualları — sürətli cavablar |
| 8. Uyuşmazlıq (disagreement) qaydası | 17. Terminlər lüğəti |
| 9. Qərar qatı: SPRT, konformal, təzəlik | |

1. Problem və həll ideyası

1.1 Problem

Onlayn ödəniş formaları avtomatlaşdırılmış skriptlərlə hücumə məruz qalır. Ən yaygın üç ssenari:

- **Kart sınaması (card testing)** — oğurlanmış minlərlə kart nömrəsi kiçik məbləğlərlə yoxlanılır.
- **Credential stuffing** — sızmış istifadəçi adı/parol cütləri kütləvi sınaq.
- **Skriptli checkout** — məhdud məhsulları bot sürətilə almaq.

Ənənəvi müdafiə **nəyin** göndərildiyinə baxır: kart nömrəsi, IP, cihaz parmaq izi. DeepCheck isə **səhifənin necə istifadə olunduğuna** baxır.

1.2 İdeya

İnsan və bot səhifə ilə fərqli qarşılıqlı əlaqə qurur:

Siqnal	İnsan	Bot / skript
Siçan yolu	Əyri, titrək, dəyişkən sürət	Düz xətt, sabit sürət, ya da ümumiyyətlə yox
Hərəkətdən əvvəl duraksama	200–1500 ms, qeyri-müntəzəm	Sıfıra yaxın, ya da mükəmməl müntəzəm
Yazma ritmi	Qeyri-bərabər	Sabit interval
Kliklər	Az sayda, aralı	Çox, bərabər aralı, piksel-dəqiq
Scroll	Dəyişən sürətli partlayışlar	Yoxdur, ya da sabit
Tab fokusu	Bəzən itir	Heç vaxt

DeepCheck bu fərqləri **12 rəqəmə** çevirir, kiçik bir model ansamblına verir və istifadəçi səhifədədirsə **hər 2 saniyədə** 0–100 arası risk skoru qaytarır.

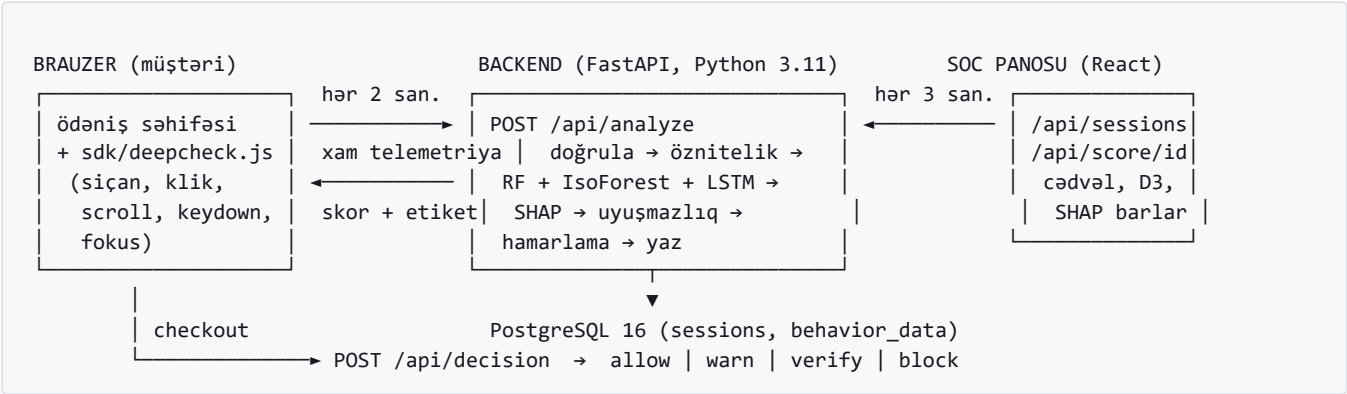
$$\text{Risk Skoru} = 100 \times P(\text{fraud} \mid \text{davranış})$$

1.3 Niyə davranış, niyə CAPTCHA yox

CAPTCHA	Konversiyanı aşağı salır, istifadəçidən əməl tələb edir, həll xidmətləri ilə aşılır.
Cihaz parmaq izi	Saxtalaşdırıla bilir, brauzer yeniləmələri ilə dəyişir.
Davranış	Passiv toplanır, istifadəçi heç nə hiss etmir, altı müstəqil siqnalı eyni anda inandırıcı saxtalaşdırmaq çətindir.

Vacib: davranış analizi digər qatları **əvəz etmir**, onlara **əlavə olunur**. Bu sənədin 14-cü bölməsi bunun niyə belə olmalı olduğunu ölçülmüş rəqəmlərlə izah edir.

2. Ümumi arxitektura



2.1 Texnologiya seçimləri və səbəbləri

Hissə	Seçim	Niyə məhz bu
Backend	FastAPI (Python 3.11)	Async I/O ilə tipli validasiya bir yerdə. Pydantic ilə sərhəddə tip yoxlaması pulsuz gəlir — telemetriya düşmənin nəzarətindədir, ona görə bu kritikdir.
ML	scikit-learn + PyTorch	Cədvəl datasında meşələr sürətli və dayanıqlı; SHAP onları nativ dəstəkləyir. Ardıcılıq modeli üçün PyTorch təbii evdir.
Baza	PostgreSQL 16	JSON sütunları + unikal indeks + advisory lock. Sonuncu ikisi replay müdafiəsi və saxlama təmizliyi üçün lazımdır.
Real-time	REST polling (2 san.)	WebSocket əlavə mürəkkəbliyə gətirərdi; 2 saniyəlik pəncərə onsuz da modelin gözlədiyi vahiddir.
Frontend	React + Vite + Tailwind	Sürətli build, dark tema, komponent təkrar istifadəsi.
Qrafik	D3.js	Risk tarixçəsi üçün tam nəzarət; hazır kitabxana lazımsız çəki olardı.
Deploy	Docker Compose	Bir əmrə hər şey qalxır, oflayn işləyir, banklar da containerlə deploy edir.

3. Brauzer SDK-sı (sdk/deepcheck.js)

Asılılığı olmayan IIFE. `window.DeepCheck = { init, stop, getSessionId, getToken, ready }` ixrac edir.

3.1 Nə dinləyir

Hadisə	Saxlanılan	Niyə
<code>mousemove</code>	<code>{x, y, t}</code>	Yolun forması və təcil
<code>click</code>	<code>{x, y, t}</code>	Klik sıxlığı və müntəzəmliyi
<code>scroll</code>	<code>{scrollY, t}</code>	Scroll sürətinin variansı
<code>keydown</code>	yalnız <code>{t}</code>	Yazma ritmi. Basılan düymə heç vaxt oxunmur.
<code>visibilitychange</code>	<code>t</code>	Fokus itkisi sayı

Məxfilik qərarı: `onKeyDown` yalnız zaman damğası yazır. Kod şərhində açıq yazılıb ki, ora `e.key` və ya sahə dəyəri əlavə edilməməlidir. Kart nömrəsi, CVV, ad — heç biri brauzerdən çıxmır. Bütün dinləyicilər `passive` -dir, yəni səhifəni ləngitmirlər.

3.2 Duraksama (hesitation) məntiqi

Hər izlənən hadisə `recordHesitation()` çağırır. Əvvəlki hadisədən bəri keçən fasilə `400 ms` -dən çoxdursa, `{gap, t}` kimi yazılır.

Əlavə incəlik: istifadəçi sadəcə **susursa**, yeni hadisə gəlmədiyi üçün fasilə heç vaxt «bağlanmır». Ona görə hər flush-da gözlənilən sükut ayrıca yoxlanılır və qeydə alınır; `lastEventAt` irəli çəkilir ki, eyni sükut iki dəfə sayılmasın.

3.3 Sürüşən pəncərə və göndərmə

Taymer hər `2000 ms` işə düşür. Hər dəfə:

- Bütün buferlər son **10 000 ms**-ə kəsilir (`ROLLING_WINDOW_MS`). Buferlər **təmizlənmir** — 2 saniyə sükut bütün öznitelikləri «data yoxdur»a sıfırlamasın deyə.
- Hər bufer serverdəki limitə uyğun kəsilir: siçan 2000, klik 500, scroll 1000, duraksama 500, fokus 200, düymə 1000. Ən **yeni** qeydlər saxlanılır.
- Heç nə toplanmayıbsa, sorğu ümumiyyətlə göndərilmir.
- `POST /api/analyze` + `X-DeepCheck-Token` başlığı.
- 2xx olmayan cavab və ya pozuq gövdə **xəta sayılır** — `onError` işə düşür, `deepcheck:error` DOM hadisəsi atılır.

Niyə vacibdir: əvvəllər xəta cavabı da JSON olduğu üçün `onUpdate` -ə ötürülürdü; istifadəçi kodu `risk_score` oxuyub `undefined` alır və təmiz skora keçirdi. Yəni **ölü backend «Gerçek Kullanıcı» kimi görünürdü**. İndi uğursuz sorğu heç vaxt `onUpdate` -ə çatmır — sistem **bağlı istiqamətdə** uğursuz olur.

3.4 Sessiya kimliyi və attestasiya

Sessiya id-si **brauzerdə yaradılmır**. İki addımlı proses:

```
1) POST /api/session      → { session_id, challenge, difficulty_bits }   (jeton YOXDUR)
2) brauzer: proof-of-work həll edir + runtime ölçmələri aparır
3) POST /api/session/attest → { session_id, token, attested: true }
```

Səbəb: müştəri öz id-sini seçə bilsəydi, başqasının sessiya id-si altında telemetriya göndərə, ya da checkout-da heç vaxt davranış göndərmədiyi id-ni təqdim edə bilərdi.

Dinləyicilər **qeydiyyatdan əvvəl** qoşulur, ona görə ilk 2 saniyənin davranışı itmir — sadəcə jeton gələnə qədər göndərilir, sürüşən bufer sayəsində növbəti flush onu təkrar göndərir.

3.5 Proof-of-work və runtime ölçmələri

Proof-of-work

SHA-256(challenge + nonce) başında **12 sıfır bit**. Gözlənilən iş ≈ 4096 heş. Chromium-da ölçdük: **7 639 heş, 75 ms**.

Saat çözünlüüyü

Ardıcıl `performance.now()` çağırışları arasında ən kiçik sıfırdan böyük fərq (mikrosaniyə). Chromium-da ölçdük: **dəqiq 100.0 μ s** — bu Chrome-un sənədləşdirilmiş clamp dəyəridir.

Taymer gecikməsi

`setTimeout(..., 0)` -un median gecikməsi. Chromium-da: **0.1 ms**. Heç bir real event loop sıfırda planlaşdırmır.

Bunlar nə sübut edir, nə sübut etmir. Sübut edir: kod **həqiqətən bir brauzer runtime-ında icra olunub**. Sübut etmir: klaviaturanın arxasında insan var. Playwright ilə sürülən real brauzer həqiqi proof-of-work və həqiqi taymer dəyərləri istehsal edir. Yəni bu qat **SDK-nı heç işlətmədən JSON göndərən** skriptləri bağlayır, brauzer avtomatlaşdırmasını yox.

Qeyd: `crypto.subtle` yalnız təhlükəsiz kontekstdə (HTTPS və ya localhost) mövcuddur. Olmadıqda qeydiyyat uğursuz olur və host səhifə `onError` ilə bağlı istiqamətdə davranır.

4. API endpointləri

Endpoint	Auth	Vəzifə
POST /api/session	—	İmzalı challenge verir (jeton yox)
POST /api/session/attest	—	PoW + runtime yoxlayıb jeton verir
POST /api/analyze	X-DeepCheck-Token	Davranış pəncərəsini skorlayır
POST /api/decision	X-DeepCheck-Token	Yeganə icra nöqtəsi — aksiyanı qaytarır
POST /api/demo/charge	X-DeepCheck-Token	Demo tacir arxa ucu: qərarı tətbiq edib təhsil edir/rədd edir
POST /api/demo/verify	X-DeepCheck-Token	Demo əlavə doğrulama — nəticəni serverdə yazır
GET /api/score/{id}	X-Dashboard-Key	Bir sessiyanın tam tarixçəsi
GET /api/sessions	X-Dashboard-Key	Son 200 sessiya (pano üçün)
GET /api/health	—	Sistem və model vəziyyəti

4.1 Giriş validasiyası — niyə bu qədər sərt

Telemetriya düşmənin nəzarətindədir. Pydantic modelləri riyaziyyata çatmadan hər şeyi yoxlayır:

Zaman damğaları	<code>int</code> , <code>[0, 4102444800000]</code> (2100-cü il)
Koordinatlar	<code>float</code> , <code>[-1e5, 1e5]</code> , <code>allow_inf_nan=False</code>
Duraksama	<code>[0, 3600000]</code> ms
Siyahı uzunluqları	SDK limitləri ilə eyni (2000/500/1000/500/200/1000)
Naməlum açarlar	<code>extra="ignore"</code> — köhnə SDK build-i sessiyanı kor etməsin

Bu niyə əlavə edildi (real hadisə): `list[dict]` hər şeyi qəbul edirdi. İki fərqli uğursuzluq baş verirdi. **Sərt çökmə:** `{"t": "abc"}` çıxma əməliyyatına çatıb `TypeError` → 500. **Səssiz korlanma:** JSON-un `NaN` / `Infinity` literalları (Python-un `json` modulu bunları qəbul edir) `np.mean` → `np.clip` → `NaN` risk skoruna axırdı; `/api/analyze` həmin sətiri Postgres-ə **yazır**, sonra cavabı serializasiya edə bilmirdi — nəticədə həmin sessiyanın bütün sonrakı oxunuşları **həmişəlik** pozulurdu.

4.2 /api/analyze emal ardıcılığı

1. Jeton yoxlanışı (`hmac.compare_digest` — vaxt sızmasına qarşı).
2. Rate limit (sessiya başına 60/dəq).
3. Sessiyanın son flush-ları oxunur — LSTM tarixçəsi, hamarlama və replay yoxlanışı üçün.
4. **Replay müdafiəsi** (aşağıda 10.2).
5. `compute_risk` **threadpool**-da işləyir — 50 ms CPU işi event loop-u bloklamasın.
6. Atomik get-or-create: `INSERT ... ON CONFLICT DO NOTHING`.
7. Median hamarlama + uyğunsuzluq baypası.
8. Sessiya sətiri yenilənir, `behavior_data` sətiri yazılır.

ThreadPool niyə: `compute_risk` ≈ 50 ms saf CPU-dur (sklearn + SHAP + torch). Birbaşa async handler-də çağırılsa, tək event loop thread-ini bloklayır — 20 paralel flush-da **1.5 saniyəyə qədər** event loop dayanması ölçüldü, bir worker ~ 13 sorğu/saniyəyə düşürdü.

Atomik insert niyə: əvvəlki «oxu-sonra-əlavə et» nümunəsi eyni yeni sessiya üçün iki paralel flush gələndə `IntegrityError` → 500 verirdi. Bu nadir hal deyil: SDK hər 2 saniyədə göndərir və ilk sorğu model isinməsini ödəyir, yəni sessiya başlanğıcında üst-üstə düşmə **adi haldır**.

4.3 Cavab formatı

```
{
  "session_id": "uuid",
  "risk_score": 73.4,           // hamarlanmış sessiya skoru
  "label": "Yüksek Risk",
  "confidence": 0.91,
  "shap_explanation": [],      // BOŞ — bax 10.5
  "response_time_ms": 18.5
}
```

5. 12 öznitelik — hər biri ayrıca

Kanonik ad və sıra `backend/lstm_model.py` -dakı `FEATURE_NAMES` -dədir. Scorer, train_model və SHAP etiketləri hamısı oradan oxuyur — bu, iki modulun öznitelik sayı/sırası üzrə bir-birindən ayrılmasının qarşısını alır.

5.1 İlk 6 — marjinal statistikalar

#	Ad	Nə ölçür	İnsan	Bot
1	<code>scroll_hizi_varyansi</code>	Scroll sürətinin variansı (px/ms)	Yüksək	0 və ya çox kiçik
2	<code>tereddut_skoru</code>	Orta duraksama (ms)	0.3–0.8	≈ 0
3	<code>etkilesim_entropisi</code>	Hadisə aralıqlarının entropiyası	0.7–1.0	≈ 0
4	<code>ivme_degisimi</code>	Siçan təcilin variansı	Orta	≈ 0
5	<code>tiklama_yogunlugu</code>	Son 5 saniyədəki klik sayı ÷ 10	Aşağı	Yüksək
6	<code>odak_degisimi</code>	Fokus itkisi sayı ÷ 5	Bəzən > 0	0

Üç kritik dizayn qərarı

(a) Entropiya kanal başına ölçülür, birləşdirilmiş axında yox. Üç müstəqil müntəzəm kanalı (siçan 80 ms, klik 150 ms, scroll 90 ms) bir axına qarışdırmaq, hər biri ayrılıqda mükəmməl robotik olsa belə, qeyri-müntəzəm görünən fasilə ardıcılığı verir — bu, fərqli periodların birləşməsindən yaranan *beat-frequency* artefaktıdır. Ölçdük: üç sıfır-entropiyalı kanal birləşdiriləndə ≈ **0.92** göstərdi. İndi hər kanal ayrıca ölçülür və fasilə sayına görə çəkili orta alınır.

(b) Sürət deltası yox, təcil. Sabit sürətli hərəkətin təcil variansı sürətdən asılı olmayaraq sıfırdır; insan hərəkətində isə həmişə nəşə var.

(c) Neytral geri-dönüş dəyərləri. Öznitelik hesablanı bilmirsə (2–3 nümunədən az), **0.0 yox**, insan və bot orta dəyərlərinin **orta nöqtəsi** qoyulur. Səbəb: 0.0 hər öznitelikdə **bot** ucundadır, yəni «data yoxdur» halı «botdan da şübhəli» kimi skorlanırdı. Bu dəyərlər **təlim zamanı hesablanır** və `model.pkl` içində saxlanılır — əl ilə saxlanılarda bir dəfə köhnəlmişdi.

`tiklama_yogunlugu` və `odak_degisimi` qəsdən istisnadır: onlar saylardır, sıfır real müşahidədir («heç klik olmayıb»), çatışmayan ölçmə deyil.

5.2 Sonrakı 6 — struktur öznitelikləri

Bunlar **ölçülmüş bir zəiflikdən sonra** əlavə edildi: düz xətlə bot yoluna cəmi **2 piksel** Gauss küyü əlavə etmək risk skorunu 54.7-dən 27.0-a endirir və qərarı «rədd»dən «qəbul»a çevirirdi. Yəni ilk 6 öznitelik «yol titrəyirmi?» sualına enmişdi və titrəmə pulsuzdur.

Ad	Nə ölçür	Niyə saxtalaşdırmaq çətindir
hiz_otokorelasyonu	Sürətin öz əvvəlki dəyəri ilə lag-1 korrelyasiyası, <code>[-1,1]</code> → <code>[0,1]</code>	Real hərəkətdə impuls var (sürətlən → pik → yavaşla). Müstəqil küy ≈ 0 verir.
yon_tutarlilikigi	Ardıcıl hərəkət vektorları arasında orta kosinus	Hədəfə yönəlmiş hərəkət eyni istiqaməti saxlayır (yüksək); küy hər addımda istiqaməti yenidən atır (≈ 0.5); düz skript heç dönmür (≈ 1).
zaman_kuantasyonu	Fasilələrin ən çox təkrarlanan millisaniyə dəyərinin payı	Skriptli taymer (<code>t += 80</code> , <code>setInterval</code>) eyni fasiləni təkrar-təkrar verir → ≈ 1 . İnsan girişi real avadanlıq taymerindədir, praktikada heç vaxt eyni ms təkrarlanmır → $\approx 1/n$.
duraklama_dagilimi	Fasilələrin variasiya əmsalı (std/orta) ÷ 1.5	İnsan fasilələri ağır quyruqludur (əsasən sürətli, bəzən çox uzun) → yüksək. Sabit gecikmə ≈ 0 ; <code>uniform(a,b)</code> gecikmə isə tavanda məhduddur.
tiklama_onesi_hareket	Öncəsində siçan hərəkəti olan kliklərin payı (1500 ms pəncərə)	Kanallar arası. İnsan kursoru hədəfə aparır, sonra klikləyir. <code>element.click()</code> heç nə hərəkət etdirmir.
kanal_gecis_gecikmesi	Klaviatura↔siçan keçidlərinin median gecikməsi ÷ 800 ms	Kanallar arası. Əl hərəkət etmək üçün real vaxt xərcləyir; <code>dispatch</code> növbələşdirən skript heç nə ödəmir.

Kiçik-nümunə qapıları — bunlar düzgünlük tələbidir, tənzimləmə deyil

<code>MIN_AUTOCORRELATION_SAMPLES = 8</code>	≥ 9 trayektoriya nöqtəsi
<code>MIN_DIRECTION_SAMPLES = 6</code>	≥ 8 trayektoriya nöqtəsi
<code>MIN_TIMING_GAPS = 5</code>	Kanal başına fasilə
<code>MIN_CLICKS_FOR_MOTION_RATIO = 2</code>	Tək klik yalnız 0.0 və ya 1.0 verir — çox kobud
<code>MIN_CHANNEL_TRANSITIONS = 5</code>	Keçid sayı

Dörd sürət nümunəsindən hesablanan avtokorrelyasiya öz seçmə xətası ilə üstünlük təşkil edir. Onu modelə «ölçmə» kimi vermək — əsasən yazan istifadəçini **85.7 «Bot Tespit Edildi»** kimi skorlamaq deməkdir. Bu, real baş vermiş yanlış pozitivdir. Həddin altında **ölçmə yoxdur**, ona görə neytral dəyər tətbiq olunur.

6. Normalizasiya: niyə log-persentil

6.1 Əvvəlki vəziyyət və problem

Üç «ağır quyruqlu» öznitelik (iki varians və bir orta müddət) əl ilə seçilmiş sabitlərə bölünüb `[0,1]` -ə kəsilirdi: `varians/5.0`, `ms/1500.0`, `varians/2.2e-6`. Ölçükdə hər üçü **tavanda** oturmuşdu:

Mənbə	ivme_degisimi
Bizim insan modeli	1.000 (kəsilmiş)
Bezier yollu bot	1.000 (kəsilmiş)
Düz xətlı bot	0.002

Yəni öznitelik faktiki olaraq **binar** idi. Real brauzer axınlarında da 6 öznitelikdən **3-ü** 1.000-də qapanmışdı.

6.2 Həll

```
norm = clip( (log10(xam) - lo) / (hi - lo), 0, 1 )  
  
lo, hi = təlim paylanmasının 1-ci və 99-cu persentili (log10 fəzasında)
```

Bu uclar **təlim zamanı ölçülür** və `model.pkl` içində `feature_scaling` kimi saxlanılır. Hazırkı modeldəki dəyərlər:

<code>scroll_hizi_varyansi</code>	log10 aralığı (−4.595, −0.342)
<code>tereddut_skoru</code>	(2.624, 3.276) → təxminən 420 ms – 1888 ms
<code>ivme_degisimi</code>	(−7.021, −5.532) → 9.5e−8 – 2.9e−6

Diqqət: köhnə bölən **2.2e−6** idi, ölçülmüş 99-cu persentil isə **2.9e−6**. Yəni bölən real aralığın **tam təpəsində** oturmuşdu — buna görə demək olar hər şey kəsilirdi. Persentil seçildi ki, tək bir qeyri-adi sessiya miqyası təyin edə bilməsin; log10 seçildi çünki kəmiyyətlər multiplikativdir.

6.3 Entropiyanın binləri

Əvvəl entropiya hər sessiyanın öz `[min, max]` aralığında binlənirdi. Nəticə: oxumaq üçün bir dəfə dayanan insan aralığı 10 ms-dən 5 saniyəyə qədər genişləndirir, bütün adi fasilələr birinci binə düşür və entropiya sıfıra çökür — halbuki metronom kimi bot dar aralıq saxlayıb **daha yüksək** skor alır. Ölçükdə: insan modeli **0.174**, headless skript **0.548** — tam tərsinə.

İndi sabit **log-millisaniyə şəbəkəsi** ($10^{0.5} - 10^4$ ms, 14 bin) istifadə olunur, yəni rəqəm hər sessiyada eyni şeyi bildirir.

Uyğunluq qaydası: `feature_scaling` olmayan bundle **rədd edilir** (`ModelUnavailableError`), çünki köhnə bölənlərlə təlim olunmuş model yeni normalizasiya ilə **öyrənmədiyi koordinat sistemini** oxuyar və inamla mənasız skor verir. Eyni məntiqə `feature_names` uyğunsuzluğu da rədd edilir.

7. Üç model və ansambl

Model	Konfigurasiya	Rolu	Çəki
Random Forest	200 ağac, dərinlik 12, min yarpaq 5	Nəzarətli təsnifat — əsas signal və izah edilə bilən nüvə	0.5
Isolation Forest	200 ağac, contamination 0.05, yalnız insan sətirləri	Nəzarətsiz anomaliya — təlimdə heç nəyə bənzəməyən davranışı tutur	0.2
LSTM	2 qat, gizli 32, dropout 0.2, Adam 1e-3, 8 epoch	10 addım × 12 öznitelik ardıcılığı — sessiya dəyişikliyi tutur	0.3

7.1 Inferens düsturu

```
scaled = StandardScaler(öznitelik vektoru)
rf_p = RF.predict_proba(scaled)[fraud]
iso_a = clip(0.5 - IsoForest.decision_function(scaled), 0, 1) # yüksək = daha anomal
lstm_p = LSTM(ardıcılıq) # son 10 flush

harman = clip(0.5*rf_p + 0.2*iso_a + 0.3*lstm_p, 0, 1)
d = |rf_p - lstm_p|
əgər d ≥ 0.35: birləşik = (1-d)*harman + d*max(rf_p, lstm_p)
əks halda: birləşik = harman

skor = round(100 × birləşik, 1)
confidence = max(p, 1-p)
```

7.2 Niyə üç model, bir yox

- **Random Forest** — cədvəl datasında dəqiq, sürətli və SHAP ilə **tam izah edilə bilən**. Jüri qarşısında «niyə bu qərar» sualına cavab verən komponent budur.
- **Isolation Forest** — simulyatorun heç təsəvvür etmədiyi hücumlar üçün. Onu **yalnız insan** sətirləri üzərində öyrədirik, yəni «normal» = insan paylanması.
- **LSTM** — tək anı yox, **trayektoriyayı** oxuyur. Sessiya ortasında dəyişən davranış (hijack) yalnız burada görünür.

7.3 LSTM həqiqətən ardıcılıq görürmü?

Bəli, indi görür. Əvvəl həm inferensdə, həm təlimdə tək öznitelik sətri 10 dəfə təkrarlanırdı — yəni LSTM ansamblın 30%-ni daşıyıb heç bir zaman məlumatı almırdı. İndi `build_sequence()` sessiyanın **real 9 əvvəlki flush-ını** Postgres-dən oxuyub cari flush ilə birlikdə verir; qısa tarixçə cari sətirlə **sol tərəfdən doldurulur**. Təlimdə də LSTM həqiqi 10 addımlıq ardıcılıqlar üzərində öyrədilir və sessiyaların **12%-i orta yerdə persona dəyişir** — modelin öyrənəcəyi yeganə zaman signalı budur.

Bunu bir test qoruyur: `test_lstm_reacts_to_trajectory` — eyni cari flush, fərqli tarixçə fərqli skor verməlidir. Verməsə, test uğursuz olur.

7.4 SHAP izahı

`shap.TreeExplainer(rf)` fraud sinfi üçün öznitelik başına töhfə verir; ən böyük 3 mütləq töhfə `impact` ($\times 100$) kimi qaytarılır. Yalnız Random Forest izah edilir: ağaclarda SHAP **dəqiqdir** (approximation deyil) və sürətlidir.

7.5 Performans qərarları

`rf.n_jobs = 1`

`n_jobs=-1` **təlim** parametridir və pickle-a yazılır. İnferensdə hər sorğu **bir** sətir proqnozlaşdırır; joblib-in thread pool qurulması işdən baha başa gəlir. Ölçüdə: **42.1 ms** → **14.4 ms** ($\approx 2.9\times$). Yükləmə zamanı təyin olunur ki, köhnə pickle-lar da düzəlsin.

`torch.set_num_threads(1)`

Tək 10 addımlıq ardıcılığın forward-u ≈ 1.4 ms — thread-lərə bölmək üçün çox azdır. Default-da hər worker nüvə başına thread yaradır və worker-lər bir-birini boğur.

Boot isinməsi

`get_bundle()` yalnız unpickle edir; ilk **real** çağırış sklearn/torch-un lazy isinməsini ödəyir ($\approx 3\times$ gecikmə). Ona görə lifespan-da bir dummy skorlama edilir.

8. Uyuşmazlıq (disagreement) qaydası

8.1 Hansı problemi həll edir

Sessiya ortasında devir-teslim (hijack): insan səhifəni açır, sonra idarəni bota verir. Ölçdük — 5 insan pəncərəsi, sonra 5 avtomatlaşdırma pəncərəsi:

Flush	Mənbə	RF	LSTM	Harman	Hamarlanmış	Qərar
5	insan	0.09	0.00	15.4	15.4	allow
6	BOT	0.99	0.01	61.2	15.4	allow
8	BOT	0.82	0.00	53.5	53.5	warn
10	BOT	0.99	1.00	91.0	61.2	verify

İki nəticə çıxdı. Birincisi: **əsas günahkar ansambl çəkiləri deyildi** — 6-cı flush-da harman onsuz da 61.2 idi, median hamarlama onu 15.4-ə basdırırdı. İkincisi: LSTM «orta» dəyər vermir, üç pəncərə boyunca **0.00** deyib sonra **1.00**-a sıçrayır.

8.2 İki dəyişiklik

(1) Uyuşmazlıq ayrıca risk termi. $d = |RF - LSTM|$ olmaqla:

$$\text{birləşik} = (1 - d) \times \text{harman} + d \times \max(RF, LSTM)$$

$d \approx 0$ -da bu köhnə harmanın eynisidir, yəni razılaşan sessiyalar toxunulmur. $d \approx 1$ -də isə **həyəcanlı komponentdir**. Qəsdən asimmetrikdir: yalnız qaldırır, endirmir — çünki hijack tutulmağa dəyən haldır və əlavə doğrulama təhsildən ucuzdur. Astana: `DISAGREEMENT_THRESHOLD = 0.35`.

(2) **Hamarlama alarmı gizlədə bilməz**. Hamarlama *tək* qeyri-adi oxunuşa qarşıdır; devir-teslim *tək* oxunuş deyil, **səviyyə sıçrayışıdır**. Kəskin uyuşmazlıqda hamarlama sıçrayışı azalda bilər, amma alarmı verən oxunuşun altına endirə bilməz.

8.3 Üçüncü təklifi niyə qurmadıq

Ayrıca `max(RF, LSTM)` tavanı istənmişdi. Uyuşmazlıq düsturu $d \rightarrow 1$ -də onsuz da ona yaxınsayır: 6-cı flush-da **99.0**, sadəcə tavan isə **99.4** verərdi. İkisini birlikdə qoymaq **eyni sübutu iki dəfə saymaq** olardı və modellərin cüzi fərqləndiyi hər skoru da qaldırardı.

8.4 Nəticə və qiyməti

Vəziyyət	6-cı flush-da qərar
Əvvəl	15.4 → allow
+ hamarlama baypası	61.2 → verify
+ uyuşmazlıq yüksəltməsi	99.0 → block

Qiyməti **commit-dən əvvəl** ölçüldü, çünki bu qayda model insan haqqında sadəcə **yanılında** da işə düşür. 40 sessiya, qayda açıq və bağlı:

Sınıf	Orta (bağlı → açıq)	Ən yüksək	Yanlış pozitiv
insan	19.5 → 19.7	32.3 → 36.7	%0
düz xətlı bot	85.3 → 85.3	—	%100 blok

Canlı API üzərindən də təsdiqləndi: insan sinfində %0 yanlış pozitiv. Taklit isə %0-dan %8-ə keçdi — 12 sessiyada 1, yəni **kü**. Bu bir hijack düzəlişidir, taklit düzəlişi deyil.

9. Qərar qatı

`POST /api/decision` — 40/60/80 pilləsinin tətbiq olunduğu **yeganə** yer. Əvvəl bu məntiq `Demo.jsx` -də, yəni hücumçunun öz brauzerində idi.

9.1 Qərar ardıcılığı

Şərt	Nəticə	reason
Sessiya sətri yoxdur	verify	unknown_session
3-dən az analiz edilmiş flush	verify	insufficient_evidence
Son flush 30 saniyədən köhnə	verify	stale
SPRT hələ qətiləşməyib	verify	insufficient_evidence / ambiguous
Skor <code>verify</code> bandında, amma 5 dəq içində doğrulama var	allow	verified
Klaster (defolt bağlı)	verify	cluster
Konformal qoruyucu blokdan yumşaldır	verify	conformal
Əks halda	pilləyə görə	score

9.2 SPRT — ardıcıl ehtimal nisbəti testi

Sabit «üç flush» qaydası mühakiməylə seçilmiş rəqəm idi. Wald-ın SPRT-si onu **sübutun nə qədər aydın olduğuna uyğunlaşan** dayanma qaydası ilə əvəz edir və verilmiş xəta nisbətləri üçün gözlənilən nümunə sayında **optimaldır**.

```
S = Σ log( p_i / (1 - p_i) )      # p_i = hər flush-ın risk skoru / 100

A = log((1-β)/α) = +4.4998      # α = 0.01 (yanlış pozitiv)
B = log(β/(1-α)) = -2.2925     # β = 0.10 (yanlış neqativ)

S ≥ A → kifayət qədər sübut, bot istiqamətində
S ≤ B → kifayət qədər sübut, insan istiqamətində
aralıqda → hələ qərar vermək olmaz → verify
```

Asimmetriya qəsdlidir: real müştərini narahat etmək satış itkisidir, bir botu qaçıрмаq bir cəhd itkisidir.

Vacib təhlükəsizlik detalı: `MIN_FLUSHES_FOR_DECISION = 3` SPRT-nin **altında** dayanır, onu əvəz etmir. Bir müddət 1-ə salınmışdı və hesab bunu ifşa etdi: aşağı hədd `-2.2925` olduğu üçün **tək** flush 9.17 və ya daha aşağı skorla artıq «insan» sərhədini keçirdi. Telemetriya hücumçu tərəfindən göndərilir və bir saxta pəncərə ən ucuz şeydir — altı saniyəlik qayda məhz bunun üçün var idi.

Qeyri-müəyyənlik heç vaxt təhsil edilmir. Əvvəl flush tavanında (10) kod pilləyə düşürdü, yəni 40–60 bandında parkda duran sessiya 10 flush-dan sonra `warn` alıb **kartı çəkirdi**. 20 saniyə qəsdən qeyri-müəyyən davranmaq təsdiq almağın yolu idi. İndi qeyri-müəyyənlik qeyri-müəyyən qalır; tavan yalnız müştəriyə deyilən mesajı dəyişir (`ambiguous`), ödənişin keçib-keçməməsini yox.

9.3 Konformal qoruyucu

Tək istiqamətli təhlükəsizlik toru. Təlim zamanı kənarda saxlanmış **real insan** sessiyalarının aldığı skorlar `model.pkl` -də saxlanılır (hazırda **36 sətir**). Yeni skorun konformal p-dəyəri:

$$p = (1 + |\{\text{kalibrasiya skorları} \geq s\}|) / (n + 1)$$

`p > 0.05` olarsa, bu skor legitim istifadəçi üçün **qeyri-adi deyil** və sistem onun üzərinə **blok qoymaqdan imtina edir**, əvəzinə əlavə doğrulama istəyir. Yalnız yumşalda bilər, sərtləşdirə bilməz — səhv olsa, qiyməti bir «challenge»-dir, müştəri deyil.

Payda və məxrəcdəki +1 standart sonlu-nümunə düzəlişidir: n kalibrasiya nöqtəsi ilə əldə edilə bilən ən kiçik p-dəyər $1/(n+1)$ -dir. 36 nümunə ilə bu **0.027**-dir — 0.05 tələbini *güclə* keçir. Təlim bunu ekranda yazır və kifayət etməyəndə xəbərdarlıq edir.

9.4 Klaster — ölçüldü və bağlı göndərildi

İdeya: taklit özünü təkrarlamaqdan qaça bilməz. Struktur özniteliklərini kobud şəbəkəyə kvantlayıb eyni «kova»da neçə **fərqli** sessiya olduğunu saymaq.

Nəticə: tutmadı. Müstəqil yazılmış insanlaşdırılmış generatorun 25 sessiyası **30 fərqli kova** verdi — 25 insan sessiyası ilə eyni yayılma. Səbəb: bacarıqlı bot öz parametrlərini hər işləmədə təsadüfiləşdirir.

Astana	Tutulan taklit	İşarələnən insan
4 qonşu	%60	%40
6 qonşu	%4	%8

Müştərini hücumçudan çox işarələyən nəzarət **heç bir nəzarətdən pisdır**, ona görə defolt bağlıdır (`CLUSTER_ESCALATION=0`). Kova hesablanmağa davam edir, çünki ucuzdur və kimlik siqnallarının (cihaz, TLS, IP) bağlanacağı təbii nöqtədir — sənaye məhz onlara görə klaster qurur, çünki onlar davranışdan qat-qat sabitdir.

Bu yolda tapılan real səhv: kova **normalizasiya olunmuş** özniteliklərdən qurulanda nəhəng bir hüceyrə yaranırdı — ölçülə bilməyən hər flush eyni neytral dəyərləri alırdı və insanlar, taklitlər, skriptlər birlikdə düşürdü. Ən böyük klasterdə **37 sessiya** var idi və hər iki sinifdən bərabər. İndi kova yalnız **ən azı 4 öznitelik həqiqətən ölçüləndə** mövcud olur.

10. Təhlükəsizlik qatları

10.1 Sessiya jetonu

`token = HMAC-SHA256(DEEPCHECK_SECRET, session_id)` . Müştəri jetonu saxlaya bilər, amma **ona verilməyən id üçün jeton düzəldə bilməz**. Müqayisə `hmac.compare_digest` ilədir — adi `==` ilk fərqli baytda dayanır və prefiks uzunluğunu vaxt vasitəsilə sızdırır.

`DEBUG=0` olduqda `DEEPCHECK_SECRET` və `DASHBOARD_KEY` təyin edilməyibse proses **başlamır**. Səssiz defolt heç vaxt olmamalıdır — çünki o, «autentifikasiya faktiki olaraq bağlıdır» vəziyyətinə səssizcə enərdi.

10.2 Replay müdafiəsi — üç yoxlama

Hücum: bir real insan sessiyasını bir dəfə qeyd et, sonra hər saxta checkout-dan əvvəl eyni flush-ları təkrar göndər. Skor insan çıxır, jeton keçərlidir, qərar «allow» olur. Bu, 20 sətirlik skriptlə bütün sistemi məğlub edir və heç bir ML qaçırma tələb etmir.

Yoxlama	Qayda	Nəyi bağlayır
Saat sapması	Ən yeni hadisə server saatından 15 saniyədən çox uzaqdırsa → 422	Qeyd, tərifinə görə köhnədir
İrəli zaman	Sessiya daxilində hər flush-ın ən yeni hadisəsi əvvəlkindən köhnə ola bilməz	Təkrar oynadılan pəncərə
Barmaq izi	Zaman damğaları ilk hadisəyə görə yenidən əsaslandırılıb SHA-256 alınır; unikal indeks	Saati «indi»yə sürüşdürülmüş qeyd

Barmaq izi axtarışı **bütün sessiyalar üzrə qlobaldır** — təzə jeton bir qeydi «yumaq» üçün kifayət etmir. İndeks **UNIQUE**-dir, çünki oxu-sonra-yaz yoxlaması yarış şəraitində paralel eyni flush-ların hamısını buraxırdı; Postgres bu yarış həll edən yeganə yerdirdi və handler `IntegrityError` -u eyni 422-yə çevirir.

10.3 Sürət limitləri

Kova	Açar	Limit (worker başına)	4 worker-də
<code>session</code>	IP	10 / dəq	≈ 40 / dəq
<code>analyze</code>	sessiya id	60 / dəq	SDK 30/dəq göndərir
<code>decision</code>	sessiya id	20 / dəq	≈ 80 / dəq

Skorlama **IP-yə görə deyil, sessiyaya görə** limitlənir: demo stendi və ya ofis çoxlu legitim istifadəçini bir IP arxasına yığır, IP limiti hamını kor edərdi. Jeton verilməsi isə IP-yə görədir, ona görə ikisi **birlikdə** işləyir.

Sayğaclar hər worker-in yaddaşındadır. Bu, açıq şəkildə sənədləşdirilib: 4 worker ilə faktiki tavan konfigurasiyanın 4 mislinə qədərdir və restart-da sıfırlanır. Sui-istifadəyə qarşı qəbul ediləndir, kvota üçün deyil. Bir neçə host olsa, paylaşılan backend lazımdır.

10.4 Saxlama müddəti (retention)

Xam telemetriya	1 saatdan sonra silinir (öznitelik və skor qalır)
Bütün sətir	24 saatdan sonra silinir
Süpürgə	Hər 10 dəqiqədə, Postgres advisory lock altında — 4 worker-dən yalnız biri işləyir

Səbəb ikiqatdır: disk və məxfilik. Real insanların davranış qeydləri süresiz saxlanılmamalıdır.

10.5 SHAP artıq müştəriyə qaytarılmır

`/api/analyze` **qiymətləndirilən tərəfə** cavab verir. Onu məhkum edən üç özniteliyi adlandırmaq hücumçuya **köklənmə signalı** verir: göndər → səbəbi oxu → dəyiş → təkrarla. Bu, canlı detektora qarşı nəzarətli optimallaşdırma döngüsüdür və bizim adversarial sınaqda **məhz bundan istifadə edildi** — insanlardan aşağı skor alan bot beləcə quruldu. İzah hər sətirdə saxlanılır və SOC panosu onu `GET /api/score/{id}` -dən (`X-Dashboard-Key` arxasında) oxuyur. Canlı nümayiş üçün `SHAP_IN_ANALYZE=1`.

10.6 Digər qatlar

- **Pano açarı bundle-a yazılır.** Bundle-a kompilyasiya edilən açar hər ziyarətçiyə verilir — «view-source» ilə aşılır. Analitik onu yazır, açar yalnız `sessionStorage` -dədir.
- **Demo endpointləri** (`/api/demo/*`) `DEBUG` -a bağlıdır. Onların doğrulama kodu səhifədə yazılan sabitdir.
- **Bütün portlar loopback-dədir** (`127.0.0.1`). `DEBUG=1` defolt olduğu üçün nəşr olunmuş inkişaf sirləri qüvvədədir; bu iki defolt yalnız port şəbəkədən əlçatan olmadıqda birlikdə təhlükəsizdir.
- **Traceback heç vaxt qaytarılmır** — həm log daşqını, həm barmaq izi orakuludur.
- **CORS** allowlist dəstəkləyir; `*` ilə credentials **bağlıdır**, çünki CORS spesifikasiyası bunu qadağan edir.

11. Verilənlər bazası sxemi

sessions		behavior_data	
id	TEXT PK (uuid)	id	SERIAL PK
created_at	TIMESTAMPTZ	session_id	TEXT FK → sessions.id
last_seen_at	TIMESTAMPTZ (idx)	created_at	TIMESTAMPTZ
risk_score	FLOAT 0..100 (check)	mouse_trajectory, click_timing,	
label	TEXT	scroll_rhythm, hesitation_intervals,	
confidence	FLOAT	focus_changes, key_events	JSON
shap_explanation	JSON	12 öznitelik sütunu	FLOAT
response_time_ms	FLOAT	risk_score	FLOAT 0..100 (check)
verified_at	TIMESTAMPTZ	payload_hash	VARCHAR(64) UNIQUE idx
		newest_event_at	BIGINT
		behavior_bucket	VARCHAR(32) idx
		client_signals	JSON
		raw_purged	BOOLEAN
		idx (session_id, created_at)	

sessions.risk_score

Hamarlanmış verdikt — badge, pillə, siyahı bunu göstərir

behavior_data.risk_score

Xam flush skoru — panodakı tarixçə qrafiki bunu çəkir

raw_purged

Açıq bayraq, «JSON boşdurmu?» yox — çünki yalnız klaviatura ilə işləyən flush-ın `mouse_trajectory` -si legitim boşdur və Postgres-də `json` tipi üçün bərabərlik operatoru yoxdur

client_signals

Yalnız qeyd olunur, skorlanmır — bax 14.3

Sxem miqrasiyası: `init_db` boot-da idempotent `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` siyahısını icra edir, yəni köhnə volume ilə işləyən quraşdırma sınıdır. Bu bir aralıq həlldir; düzgün cavab Alembic-dir.

12. Model təlimi

12.1 Sintetik data

python train_model.py (seed 42, tam təkrarlana bilən):

1. **25 000 sessiya** simulyasiya olunur, hər biri **10 ardıcıl flush pəncərəsi** — cəmi 250 000 öznitelik sətri.
Personalalar:
 - human — titrək əyri siçan (fasilə 50–150 ms), 400–1200 ms duraksama, 10–35 qeyri-bərabər düymə, scroll partlayışları
 - human_rushed (insanların %10-u) — tələsən real insan: az duraksama, 30% halda demək olar yalnız yazma
 - bot — headless: 0–3 siçan nöqtəsi və ya 80 ms ± 10 düz xətt, hər 150 ms ± 8 klik, hər 3 ms düymə
 - bot_sophisticated (botların %10-u) — insan aralıqlarını təqlid edir, amma **çox hamar**
2. Sessiyaların **%12-i orta yerdə persona dəyişir** (DRIFT_RATE) — ardıcıl model üçün öyrəniləcək yeganə zaman siqnalı budur.
3. Hər simulyasiya edilmiş yük **istehsalatda işlədilən eyni extract_features()** -dən keçirilir. Bu, boru xəttindəki ən vacib dizayn qərarıdır: təlim və xidmət **bir kod yolunu** paylaşır, ona görə öznitelik səhvi birində olub o birində olmaya bilməz.
4. 80/20 stratifikasiyalı bölmə, StandardScaler yalnız train üzərində.
5. RF və IsoForest **son pəncərə** üzərində; LSTM **həqiqi 10 addımlıq ardıcılıq** üzərində.

%10 «çirkəlmə» personaları qəsdlidir: siniflər asanlıqla ayrılan olsaydı, bu bir modelləmə qoxusudur, nəticə deyil.

12.2 Real brauzer datası

lab/capture.py **real Chromium** brauzerini real SDK vasitəsilə sürür və etikətlənmiş telemetriya yazır. Hazırda **234 nümunə**, 6 ssenari:

Ssenari	Etiket	Nümunə
H1_human	insan (0)	50
H2_keyboard_only	insan (0)	44
A1_naive	bot (1)	16
A2_randomized	bot (1)	27
A3_human_mimic	bot (1)	40
A4_evasive	bot (1)	57

Bunlar sintetik dataya **çəki 120** ilə qarışdırılır (real sətirlər say etibarilə çox azdır, çəkisiz onlar yuvarlaqlaşdırma xətası olardı). Kənarı saxlama **run səviyyəsindədir**, flush səviyyəsində yox — bir sessiyanın flush-ları bir-biri ilə korrelyasiyalıdır və təsadüfi flush bölməsi eyni sessiyanı hər iki tərəfə qoyub **təkrarlanmayan** rəqəm verərdi.

Bu labın tapdığı ən vacib fakt: eyni qaçamaq hücumu simulyatorada **88.9 (bloklandı)**, real Chromium üzərindən isə **36.5 (onaylandı)** aldı — çünki iki ağır zamanlama özniteliyi iki paylanma arasında **tərsinə** işləyirdi (etkilesim_entropisi : real bot 0.225, sintetik bot 0.836). Simulyator təkbaşına zamanlama özniteliklərində **yanlış işarəni** öyrədir.

12.3 Real insan sessiyalarının qeydi

```
cd backend
python record_session.py --list
python record_session.py --label human --to-training <session-id>
python train_model.py
```

`--to-training` bayrağı kritikdir: onsuz qeyd `data/real/` -a düşür, oranı yalnız `evaluate.py` oxuyur və **model heç nə öyrənmir**. Bayraqla sessiya `lab/real_telemetry.json` -a, yəni təlimin oxuduğu fayla qarışdırılır və holdout cədvəlində `R1_human_live` kimi görünür.

Toplama qaydaları `data/real/README.md` -dədir. Ən vacib üçü: subyektə bot detektoru olduğunu **demə** və «təbii davran» **istəmə** (hər ikisi davranışı dəyişir); ən azı 10–15 saniyə qarşılıqlı əlaqə; **giriş cihazını dəyişdir** — siçan kinematikası cihazdan başqa hər şeydən çox asılıdır.

13. Ölçmələr — real nəticələr

13.1 Kənarda saxlanmış real brauzer run-ları

Ssenari	n	orta p(bot)	işarələnən
H1_human (legitim)	25	0.043	%0
H2_keyboard_only (legitim)	11	0.088	%0
A2_randomized	8	0.671	%75
A3_human_mimic	10	0.927	%100
A4_evasive	28	0.599	%68

Yanlış pozitiv %0, bot recall %76.1 — bu ssenarilərə qarşı.

«Yanlış pozitiv %0»-ı diqqətlə oxuyun. H1 və H2 sətirləri real brauzerdə sürülən **skriptlənmiş** insan davranışdır, insanların qeydi deyil. Yəni %0 «detektor bu labın legitim istifadəçi modelini işarələmir» deməkdir. Bu, simulyasiya edilmiş öznitelik vektorlarına qarşı %0-dan qat-qat güclü, müştərilərə qarşı %0-dan isə qat-qat zəif iddiadır.

H2 (yalnız klaviatura) hücum sətirləri qədər vacibdir: ödəniş məhsulu üçün legitim klaviatura istifadəçisini bloklamaq bir botu qaçırmaktan pisdır və yalnız aşkarlamayı ölçən lab bunu heç vaxt üzə çıxarmaz.

13.2 Müstəqil adversarial düzənək

Yuxarıdakı ssenarilər **təlimdə var** (run səviyyəsində kənarda saxlanıb). Yəni o cədvəl «nümunəsi olan hücumun daha çox sessiyasını tanıyırmı?» sualına cavab verir. «Heç görmədiyi hücumu tanıyırmı?» sualı üçün ayrıca düzənək yazıldı — minimum-jerk trayektoriyaları, Fitts qanunu, log-normal düymə intervalları; nə simulyatorun personalarına, nə labın ssenarilərinə istinad etmədən.

Persona	Orta skor	Qərar	Dayandırıldı	AUC
insan	18.5	%100 allow	%0 yanlış pozitiv	—
naive headless skript	33.2	%100 verify	%100	0.44
düz xətlə avtomatlaşdırma	85.7	%100 block	%100	1.00
müstəqil insanlaşdırılmış bot	26.0	%91 allow	%8	0.87
müstəqil öznitelik-bilən bot	13.5	%100 allow	%0	0.02

Diqqət: **naive skript %100 dayandırılır, amma model tərəfindən yox** — 12 sessiyanın hamısı `insufficient_evidence` ilə. Headless skript formu ~1.5 saniyədə doldurur və üç flush istehsal etmir. Onun öz model skoru 33.2-dir, yəni təkbaşına `allow` deməkdir.

13.3 Titrəmə ablasyonu — düzəlişin sübutu

Eyni metronom bot, yalnız siçan küyünün standart sapması dəyişir:

Küy	Normalizasiya düzəlişindən əvvəl	Sonra
0.00 px	54.7 warn	90.4 block
1.00 px	54.4 warn	88.9 block
2.00 px	27.0 allow	88.6 block
4.00 px	27.4 allow	89.4 block

İki piksel küy əvvəl skoru yarıya endirib qərarı çevirirdi; indi onu 1.8 bal hərəkət etdirir və heç nəyi dəyişmir.

14. Nə işləmir və niyə

Bu bölmə sənədin ən vacib hissəsidir. Jüri qarşısında ən güclü mövqe iddia deyil, ölçülmüş dürüstlükdür: sistemi öz müstəqil hücum düzənəyimizlə sınaq, harada uğursuz olduğunu rəqəmlərlə bilirik və mexanizmini izah edə bilirik.

14.1 Yüksək keyfiyyətli taklit tutulmur

Müstəqil yazılmış insanlaşdırılmış bot **26.0**, insan **18.5** alır və 40 astanasının xeyli altında qalır. Öznitilik-bilən bot **13.5** ilə insan ortalamasının **altındadır**.

Səbəb struktur problemdir, tənzimləmə deyil. 12 özniteliyin hər biri müştərinin seçdiyi rəqəmlərin xülasə statistikasındır. Hədəf paylanmasını bilən bot ona dəyə bilər. Telemetriya həqiqətən insan paylanmasından çəkilibsə, **həmin telemetriyanın heç bir funksiyası onu ayıra bilməz** — bu, modelləmə çatışmazlığı deyil, bilgi-kuramsal sərhəddir.

14.2 Isolation Forest bu halda hücumçunun lehinə işləyir

IsoForest yalnız insan sətirləri üzərində öyrədilir, yəni «normal» = insan paylanması. Ansambl anomaliya skorunu `0.5 - decision_function` kimi oxuyur. Enjektə edilmiş real insan izləri bu fəzada **aykırı deyil** — tam əksinə, **ən sıx bölgədədir**. Nəticədə anomaliya termi sıfıra gedir və onun %20 çəkisi risk skorunu **aşağı çəkir**. Yəni tarafsız uğursuz olmur, hücumçunun lehinə səs verir.

14.3 Provenance siqnalları qəsdən skorlanmır

SDK hər flush-da üç «mənsə sayğacı» göndərir: `isTrusted` false olan hadisə sayı, `navigator.webdriver`, göstərici tipi (mouse/pen/touch). Bunlar saxlanılır, **amma modelə çatmır**. İki səbəb:

- Onları **qiymətləndirilən müştərinin özü bildirir**, yəni yalan deyə bilər.
- Hər biri qismən şeyi tutur: `isTrusted` səhifə JS-i tərəfindən sintez edilən hadisələr üçün false-dur, amma Playwright/Puppeteer ilə sürülən brauzer **trusted** hadisələr yaradır. `navigator.webdriver` əksinədir — sürülən brauzeri işarələyir və asanlıqla yamaqlanır.

Ona görə sütunlar var ki, real qiymətləndirmə dəsti olduqda dəyərləri **ölçülə** bilsin, fərz edilməsin.

14.4 Bu həftə ölçmədə uğursuz olan üç mexanizm

Mexanizm	Kağız üzərində	Ölçmədə
Normalizasiya düzəlişi	Taklit daha yaxşı tutulacaq	Kobud botlarda böyük qazanc; taklit bir az daha görünməz oldu (AUC 0.77 → 0.44)
6 struktur özniteliyi	Sintetik hərəkət ifşa olunacaq	Labın öz taklitini %100 tutdu, müstəqil taklitin %0-ını — yəni o botları öyrəndi, taklit anlayışını yox
Sessiyalararası klaster	Taklit özünü təkrarlayır	Hər astanada müştəriyə botdan çox zərər verdi → bağlı göndərildi

Bu ardıcılıq təsadüf deyil: problem **kateqoriyadadır**, implementasiyada yox.

14.5 Real insan datası hələ yoxdur

Modelin gördüyü bütün «insanlar» skriptdir. Real brauzer datası ilə ilk dəfə öyrədiləndə insanlaşdırılmış botun ayırd ediləbilirliyi bir təlim turunda **yazı-turadan 0.92-yə** qalxdı. **Signal oradadır**; çatışmayan şey eşikləri dürüstcə yerləşdirməyə yetəcək qədər real sessiyadır. Eşikləri öz hücum düzənəyimizə görə sürüşdürmək məhsulu **hücumçuya** uyğunlaşdırmaq olardı, istifadəçilərə yox.

14.6 Digər açıq maddələr

- **Sürət limitləri worker başındadır** — paylaşılan backend lazımdır.
- **Açar rotasiyası yoxdur** — `DEEPCHECK_SECRET` dəyişəndə bütün canlı jetonlar düşür.
- **Pano açarı paylaşılan sirdir**, per-analitik kimlik və audit izi yoxdur.
- **Sxem miqrasiya aləti yoxdur** — boot-da `ALTER TABLE` aralıq həlldir.
- **Bundle-da təlim datası mənşəyi qeyd olunmur** — start yoxlaması sklearn versiyasını və öznitelik dəstini bilir, amma modelin **nə üzərində** öyrədildiyini bilmir. Məhz buna görə bir müddət konteyner sintetik-yalnız modeli səssizcə xidmət etdi.

15. Testlər, CI və işə salma

15.1 Test dəsti — 30 test

Hər test **real baş vermiş** bir səhv və ya hücumdur; hər biri geri qayıtmasın deyə yazılıb. Testlər **davranışı** təsdiqləyir, dəqiq dəyərləri yox — yəni öznitelik düsturu və ya təlim paylanması dəyişəndə səssiz deqradasiya əvəzinə **səsli uğursuzluq** olur.

Qrup	Nəyi qoruyur
7 skarlama ssenarisi	Təbii insan aşağı; yalnız yazan insan aşağı; headless bot yüksək; skriptli hərəkət yüksək; bir təsadüfi duraksama ilə bot hələ yüksək; sürətli insan hələ aşağı; siçansız sürətli klaviatura yüksək
LSTM	Eyni cari flush + fərqli tarixçə = fərqli skor
Autentifikasiya	Yoxa/yanlış/başqa sessiyanın jetonu rədd edilir; pano açarı tələb olunur
Replay	Köhnə zaman damğası; geriye gedən zaman; saatı sürüşdürülmüş təkrar
Qərar	Ardıcıl sübut; bayat sessiya; qeyri-müəyyənlik heç vaxt təhsil edilmir
Attestasiya	Həll olunmamış PoW; sıfır taymer dəyərləri; başqa sessiyaya köçürülmüş həll
Digər	Sürət limiti + limiterin yaddaşı məhdud; eksik LSTM faylı; SHAP sızdır; klaster defolt bağlı; konformal yalnız yumşaldır

API testləri qəsdən **stub baza** ilə işləyir, Postgres ilə yox: infrastruktur tələb edən autentifikasiya yoxlaması, **sınanmağı dayandıran** yoxlamadır.

15.2 CI

`.github/workflows/ci.yml` hər push-da: kiçildilmiş datasetlə model öyrədir, test dəstini **bazasız** işlədir, frontend-i build edir və **bundle-da pano açarı varsa build-i uğursuz edir**.

15.3 İşə salma

```
docker-compose up --build

# Demo:      http://localhost:3000/demo
# Pano:       http://localhost:3000/dashboard
# API doc:    http://localhost:8000/docs
```

İlk start modeli öyrədir — makinaya görə **4–8 dəqiqə**. Nümayişdən əvvəl bir dəfə qaldırın: soyuq konteyner asılmış kimi görünür. `entrypoint.sh` Türkçə bildiriş yazır.

Frontend	Production build nginx ilə verilir. Hot-reload üçün: <code>docker-compose -f docker-compose.yml -f docker-compose.dev.yml up</code>
Model faylları	sklearn versiyasına görə adlanır (<code>model-sklearn1.5.0.pkl</code>) — pickle onu yazan versiya ilə oxunmalıdır
Konfiqurasiya	<code>.env.example</code> -a baxın: <code>DEEPCHECK_SECRET</code> , <code>DASHBOARD_KEY</code> , <code>DEBUG</code> , <code>DEMO_ENDPOINTS</code> , <code>SHAP_IN_ANALYZE</code> , <code>CLUSTER_ESCALATION</code> , <code>REAL_TELEMETRY_PATH</code> , <code>POW_DIFFICULTY_BITS</code> , retention saatları

16. Jüri sualları — sürətli cavablar

Sual	Cavab
Niyə davranış, CAPTCHA yox?	CAPTCHA konversiya itirir və həll xidmətləri ilə aşılır. Davranış passiv toplanır və altı müstəqil siqnalı eyni anda saxtalaşdırmaq çətinidir. Digər qatları əvəz etmir .
LSTM həqiqətən nə edir?	Sessiyanın real 10 flush tarixçəsini oxuyur. Sessiya ortasında dəyişən davranışı yalnız o görür. Bir test bunu qoruyur.
Yanlış pozitiv nisbəti nədir?	Test edilən hər populyasiyada %0 . Amma «insanlar» skriptdir — bu, «bu labın istifadəçi modelini işarələmir» deməkdir, «müşətiləri işarələmir» yox.
Bot brauzerdə real insan izləri enjekte etsə?	Tutmuuq və niyəsinə dəqiq bilirik. Vektor həqiqətən insandırısa heç bir öznitelik bunu dəyişmir; IsoForest bu halda hücumçunun lehinə səs verir.
50 ms ödəniş səhifəsi üçün yavaş deyil?	Çağırış asinxrondur və kritik yolda deyil. Ölçülən cavab müddəti 18–20 ms .
Hansı data toplanır? KVKK/GDPR?	Koordinatlar, zaman damğaları, scroll offseti, fokus itkisi, üç mənzə sayğacı. Düymə dəyəri, sahə məzmunu, DOM — heç biri . Təsadüfi sessiya UUID-dən başqa identifikator yoxdur. Xam qeydlər 1 saatdan sonra silinir.
Bank necə integrasiya edir?	Bir <code><script></code> , bir <code>DeepCheck.init()</code> , və öz arxa ucundan <code>POST /api/decision</code> çağırır <code>action</code> -a əməl etmək. Konteyner öz şəbəkələrində işləyir, data perimetri tərk etmir.
Necə miqyaslanır?	Bir worker $\approx$ 20 sessiya/2 saniyə. Konteyner başına 4 worker və konteynerlər state-siz, yəni üfüqi miqyaslama load balancer məsələsidir. Yeganə paylaşılan state Postgres-dir.
Piyasada sizi keçən sistemlər var, bu layihə boşunadırmı?	Xeyr . Onların üstünlüyü daha yaxşı model deyil, data miqyası və şəbəkə mövqeyidir (trilyonlarla siqnal, TLS sonlandırma nöqtəsi). Onlar da həll etməyib — sağlam bir «bypass» sənayesi var. Bizim verdiyimiz: skriptli avtomatlaşdırma %100 bloklanır, %0 yanlış pozitivlə, və üstündə istehsalat formalı icra qatı var.
Ən çətin səhv hansı idi?	Duraksama istehsalatda 2 saniyəlik, təlimdə isə $\sim$ 10 saniyəlik pəncərədə ölçülürdü. Nəticədə real flush-ların əksəriyyətində öznitelik neytral dəyərində ilişib qalırdı. Tapmaq üçün canlı trafikə öznitelik paylanması təlim dəsti ilə müqayisə etmək lazım gəldi.
Sonra nə edərdiniz?	Sırayla: real insan sessiyalarını qeyd etmək; müştərinin bildirmədiyi siqnallar (<code>getCoalescedEvents()</code> ən güclü namizəddir); qeyd kitabxanasının təkrar istifadəsini aşkarlamaq; davranış kimlik/şəbəkə/kart qatları ilə birləşdirmək.

17. Terminlər lüğəti

Termin	İzah
Flush	SDK-nın 2 saniyədə bir göndərdiyi, sürüşən 10 saniyəlik pəncərə
Öznitelik (feature)	Bir flush-dan çıxarılan 12 rəqəmdən biri
SHAP	SHapley Additive exPlanations — proqnoza öznitelik başına töhfə
Isolation Forest	Nöqtənin nə qədər asan izolyasiya olunduğunu ölçən nəzarətsiz model; asan izolyasiya = anomal
Median hamarlama	Sessiyanın rəsmi skoru son 5 xam flush skorunun medianıdır
Neytral dəyər	İnsan və bot təlim ortalarının orta nöqtəsi; öznitelik ölçülə bilməyəndə istifadə olunur
Çırkənləmə personası	Hər sinfin %10-u qarşı sinfə oxşadılır ki, model asan ayrılmadan «bəhrələnməsin»
SPRT	Sequential Probability Ratio Test — sübut kifayət edənə qədər toplayan dayanma qaydası
Konformal p-dəyər	Kalibrasiya insanların neçə faizinin bu qədər yüksək skor aldığı
Attestasiya	Kodun real brauzer runtime-ında icra olunduğunun sübutu (PoW + taymer ölçmələri)
Uyuşmazlıq (d)	RF - LSTM — modellərin fərqli zaman miqyaslarında nə qədər ayrıldığı
Hijack	Sessiya ortasında idarənin insandan bota keçməsi

Son söz. Bu layihənin ən güclü tərəfi tutduğu botlar deyil — **tutmadıqlarını bilməsi və niyəsini izah edə bilməsidir**. Yarışmalarda adi hal detektorla eyni müəllifin yazdığı simulyatordan çıxan doğruluq faizini təqdim etməkdir. Biz o tələdən çıxdıq: sistemi müstəqil yazılmış hücumçu ilə sınaq, uğursuzluğu ölçdük və sənədləşdirdik. Müdafiə edə bilməyəcəyimiz yeganə versiya — tutmadığını tuturam deyən versiyadır. Əlimizdəki o deyil.